

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 04 • NIVEL 3 - AVANZADO

Clase 04: Tool Calling y Function Calling en Python

«Dotando de Manos y Herramientas al Cerebro del LLM»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 04** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender la invocación de herramientas, esquemas de funciones y el protocolo de ejecución cliente-servidor.

Competencia Práctica: Construir un despachador dinámico que permita al LLM ejecutar funciones Python locales (clima, base de datos, APIs).

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 04: Tool Calling y Function Calling en Python

Tool Calling permite que un LLM decida autónomamente cuándo invocar funciones de código externo para consultar datos o actuar.

☀ Metáfora Central: Dotando de Manos y Herramientas al Cerebro del LLM

El LLM es un cerebro brillante pero ciego y sin manos; las herramientas son sus brazos mecánicos para interactuar con el mundo.

Principios Teóricos y Modelo Mental

El LLM NO ejecuta el código directamente: devuelve un objeto estructurado con el nombre de la función y sus argumentos.

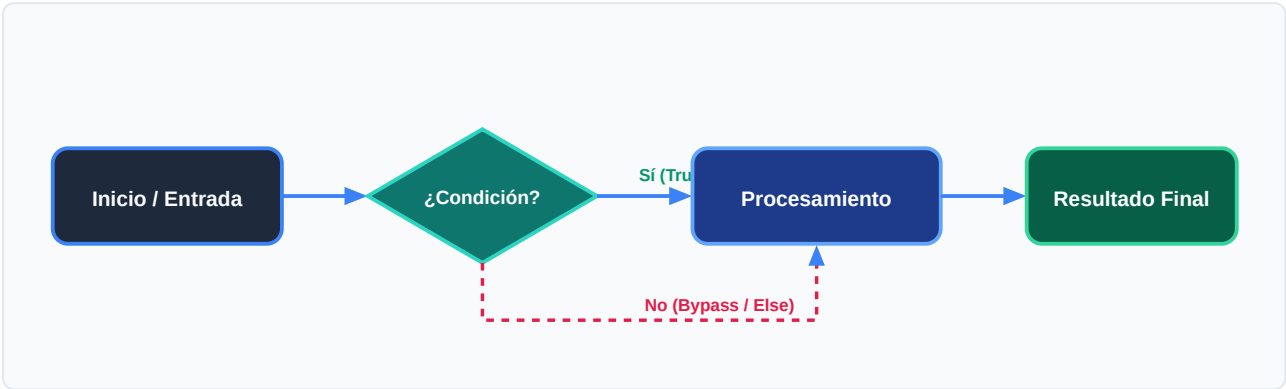
Tu backend intercepta la solicitud, ejecuta la función real en Python y le devuelve el resultado al LLM.

⚡ Regla de Oro en Python

Escribe docstrings extremadamente claros en tus funciones: el LLM los usa como manual de instrucciones.

Arquitectura y Movimiento de Datos en Memoria

Ciclo completo de Tool Calling: Solicitud -> Despacho -> Ejecución -> Respuesta.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción delintérprete	Estado en Memoria
1. Inicialización	Registro de funciones en el catálogo de herramientas.	Definiciones de herramientas cargadas.
2. Evaluación	El LLM analiza el prompt y genera un ToolCall con argumentos.	Llamada pausada esperando ejecución.
3. Transformación	Despachador local ejecuta func(**args) en Python.	Resultado calculado en backend.
4. Retorno / Salida	Envío del resultado de vuelta al LLM para formular la respuesta final.	Respuesta natural entregada al usuario.

Visualización Mental

El LLM solo decide QUÉ herramienta usar y con QUÉ argumentos; tú tienes el control de la ejecución.

Código de Demostración en Python 3.10+

Motor de despacho de herramientas dinámico en Python:

main.py (Python 3.10+)

PEP 8 Compliant

```
import math

def calcular_distancia(x1: float, y1: float, x2: float, y2: float) -> float:
    """Calcula la distancia euclidiana entre dos puntos (x1, y1) y (x2, y2)."""
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

HERRAMIENTAS = {
    "calcular_distancia": calcular_distancia
}

def despachar_herramienta(nombre: str, argumentos: dict):
    if nombre in HERRAMIENTAS:
        return HERRAMIENTAS[nombre](**argumentos)
    raise ValueError(f"Herramienta '{nombre}' no encontrada.")

res = despachar_herramienta("calcular_distancia", {"x1": 0.0, "y1": 0.0, "x2": 3.0, "y2": 4.0})
print("Resultado de la herramienta:", res) # 5.0
```

Análisis de Ingeniería del Código

Registro centralizado de funciones con type hints y ejecución segura por desempaquetado de diccionario.

Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Riesgos de seguridad en ejecución de herramientas.

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar `eval()` o `exec()` para ejecutar herramientas abre una vulnerabilidad crítica de inyección de código.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
eval(f'{nombre_funcion}({argumentos_crudos})') #  
❌ Vulnerabilidad RCE crítica
```

✅ Patrón Correcto

```
HERRAMIENTAS[nombre](**argumentos) # ✅ Mapeo  
explícito a funciones seguras
```



Consejo de Resiliencia en Producción

Implementa validaciones con Pydantic para los argumentos de cada herramienta antes de ejecutarla.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 04: Tool Calling y Function Calling en Python**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Dotando de Manos y Herramientas al Cerebro del LLM
2. Regla de Oro	Escribe docstrings extremadamente claros en tus funciones: el LLM los usa como manual de instrucciones.
3. Gotcha Crítico	Usar eval() o exec() para ejecutar herramientas abre una vulnerabilidad crítica de inyección de código.
4. Buenas Prácticas	Implementa validaciones con Pydantic para los argumentos de cada herramienta antes de ejecutarla.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea una herramienta que consulte el clima simulado de una ciudad y conéctala a un despachador.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.